

Architecture Assessment

Pipeline structure:

KG narrowing → temporal scoring → candidate generation → evidence retrieval → evidence refinement → synthesis.

Critical Issues

The TSKR index only keeps the first pattern per target_id. In `_index_tskr_patterns` (both engines):

python

if target_id and target_id not in index:

index[target_id] = p

If a failure mode has multiple TSKR patterns with different temporal relations (e.g., one precedes at short lag and one simultaneous), only the first survives. For nuclear RCA this matters because the same failure mode may have historically appeared at different latencies under different operating conditions. You need to store a list and select the highest-confidence pattern, or aggregate across patterns.

The governance score rewards PM failures, not penalizes them. In both engines:

python

return min(1.0, (failed / total) * 0.8 + (0.2 if failed > 0 else 0.0))

A candidate with zero failed PM checks gets 0.40. A candidate with all checks failed gets 1.0. The governance dimension is being treated as a positive contributor when it should function as a causal signal — high PM failure rate increases the plausibility of a maintenance-induced failure hypothesis, but this should influence the *structural or evidence score* of specific candidates linked to that PM, not uniformly boost all candidates regardless of whether they're related to the failed PM item. Right now a bearing wear candidate and an instrument drift candidate both get the same governance score boost if any PM check failed anywhere on the asset.

The evidence scoring in ChromaEvidenceRetriever operates on raw text overlap, not semantic similarity. `_assess_hit_against_candidate` uses keyword tokenization against a snippet that was already retrieved by vector similarity. This double-scores on different axes but neither axis is very discriminative. More importantly, `candidate_term_overlap` only matches the `cause_label` string against snippet tokens — so "loss of lubrication" won't match a snippet that says "lube oil degradation" even though they're semantically equivalent. For a nuclear corpus where terminology is inconsistent across documents, this will systematically under-score semantically relevant evidence.

The `_evidence_score_for_fm` and `_evidence_score_for_past_event` methods don't account for document age or relevance proximity. They score based purely on `doc_type` presence in the KG neighborhood, not on whether those documents are temporally or topologically proximate to the event. A 10-year-old SOP contributes the same as a CR filed last week. The `kg_context.json` schema has `time_distance_days` on documents — that field should feed into the evidence prior score in the causality engine.

Causality Engine — Specific Gaps vs. Engineer Needs

Symptom-to-failure-mode matching is absent. The `event.json` schema has `symptom_signature.symptom_types` and `anomaly_pattern`, and `kg_context` failure modes have `expected_symptoms` and `expected_anomaly_pattern`. Neither engine uses these fields. This is a significant missed opportunity — if the event presents as `gradual_drift` and a failure mode expects `gradual_drift`, that should produce a strong structural match signal. Right now a bearing wear candidate gets the same structural score whether the event was a gradual drift or a sudden step change.

Common cause scoring doesn't use train configuration. The `operational_context.json` has `train_configuration` with `in_service`, `oos_reason`, `oos_start`. Common cause failures in nuclear plants frequently manifest as a loss of redundancy — both trains affected by the same condition. `_common_cause_features_for_candidate` builds its cluster from KG topology but never checks whether the operational context shows multiple trains affected simultaneously, which is the clearest CCF signal available.

Recurrence analysis doesn't distinguish resolved from unresolved past events. The `kg_context.past_events` schema has a `resolved` boolean field. When `_recurrence_features_for_candidate` matches past events, it should weight unresolved or repeatedly-recurring events differently from resolved ones. A failure mode that appears three times and was never resolved is a qualitatively different recurrence signal than one that appeared three times and was corrected each time.

The temporal scorer treats all anomaly timestamps as equal. `_extract_anomaly_points` returns a flat list of datetimes with no weighting by severity, pattern type, or signal quality. A high severity `step_change` anomaly 2 hours before the event should contribute differently to `mean_lag` than a low severity spike from sensor noise. The `severity_score` field in `telemetry_summary.json` anomalies is never used in the TSKR scorer.

Retrieval Layer Issues

ProcessedRecordStore is fully in-memory with no persistence. If the process restarts, all hydration data is gone. For a tool that may need to run multiple analyses in sequence or be interrupted during an investigation, this means re-loading JSONL files each time. This is probably acceptable for a dev deployment but needs to be noted as a production gap.

_normalize_filter_meta silently drops component_ids. The comment says it's unsupported, but this means all component-level filtering in Chroma is a no-op. If the evidence retriever passes component IDs as filters expecting them to narrow the corpus, it's getting back all documents for the asset regardless of component. For a plant with 40,000 instrument channels and thousands of documents, this significantly hurts retrieval precision.

The hybrid retrieval `hybrid_weight` parameter defaults to 0.5 but BM25 is session-scoped. When `query_doc_type` is called on a collection loaded from disk (not from an in-process ingest), `state.bm25_docs` is empty and `_get_or_build_bm25` returns `None`. The call falls back to dense-only retrieval, but the caller gets no indication this happened. The `hybrid_weight` setting becomes meaningless and the RRF fusion runs on a single-view input. Add a warning or a flag in the returned documents indicating BM25 was unavailable.

Schema Builder and KG Ingest

kg_schema_builder_workflow.py uses ad-hoc label resolution that will silently create wrong-typed nodes. `_label_candidates` generates Pascal-case variants from snake_case names, which works for `processed_text_record` → `ProcessedTextRecord` but will fail silently for any label that doesn't follow this exact convention. If a TOML schema uses `AbnormalEvent` and the code generates `Abnormalevent`, it creates a new unlabeled node type rather than erroring. The label resolution needs an explicit registry lookup with a hard failure on miss.

The `causes` edge in the graph builder is overloaded across at least six semantically distinct relationships. `Anomaly` → `Event`, `PMCheck` → `Event`, `HistoricalEvent` → `CurrentEvent`, and `Alarm` → `Event` all use "causes" as the relationship type. This makes the KG unusable for path-strength analysis — when you traverse `CAUSES` edges you can't distinguish "this alarm occurred simultaneously with the event" from "this failure mode was confirmed as the root cause." Each of these needs a distinct edge type.

`_prefix` produces IDs like `FM:loss of lubrication` if the source key contains spaces. Neo4j node IDs with spaces are valid but will cause problems in Cypher when used unquoted. The `prefix` function should normalize the value portion: lowercase, replace spaces with underscores or hashes. The `_norm_text_key` helper exists in the same file but isn't used in `_prefix`.

Orchestrator Structure

rca_reasoning_orchestrator.py is doing too much. It contains the orchestrator, all concrete implementations (`Neo4jKGContextBuilder`, `HeuristicIshikawaEvaluatorV1`, `FileArtifactStore`, `DummyLLMClient`, `OllamaLLMClient`, `NoOpSchemaValidator`), and the factory function `build_dev_orchestrator`. This was flagged as a known debt item and it's worth fixing before the file grows further. At minimum, split out the KG context builder, the Ishikawa evaluator, and the factory function into separate modules.

`build_dev_orchestrator` exposes `stop_on_validation_error` but the orchestrator will raise on the first validation failure regardless of whether the error is in an optional artifact. If `ishikawa_matrix` fails schema validation (it's newly added and marked heuristic), the entire run aborts even though the RCA card could still be synthesized from the other artifacts. Validation failures on optional stages should be logged and surfaced in the manifest but should not abort the run unless the failed artifact is required by a downstream stage.

Document-type-aware treatment

The current implementation treats document types differently in some ways but not nearly as differently as their fundamentally different nature requires.

What each document type actually communicates — and what the code misses

Condition Reports (CRs)

- What they are: An engineer's observation of an abnormal condition at a specific time. They contain a symptom description, an initial assessment, and sometimes a preliminary cause statement.

- What they uniquely provide: Temporal grounding. A CR has a `created_at` timestamp and a `reported_date`. The cause statement in a CR is an *initial hypothesis*, not a confirmed finding. The CR filed on Day 9 of the precursor period in the test case ("possible fouling or CW temperature effect") was written before the DO data showed air in-leakage — it reflects the state of knowledge at that moment, not the confirmed root cause.
- What the code misses: It doesn't distinguish between a CR's preliminary cause statement and a confirmed cause statement. Both score the same. It also doesn't use the CR timestamp to assess whether the document predates the discriminating evidence — a CR written before the hotwell DO elevated should carry less causal weight than one written after.

Work Orders (WOs)

- **What they are:** Records of maintenance activities with as-found and as-left condition assessments. They contain measurements, observations, and often explicit "within normal limits" or "outside acceptance criteria" language.
- **What they uniquely provide:** The closest thing to ground truth about equipment condition at a specific point in time. WO-2024-11847 saying "zero tubes plugged, within normal limits" is a direct physical inspection result — it's qualitatively different from a CR's operator observation or a SOP's conditional guidance.
- **What the code partially handles:** The contradiction cue detection does pick up "within normal limits" and "acceptable" from WOs, which is why WO-2024-11847 correctly scores as contradicting for the fouling hypothesis in the test case. But the code doesn't know this is a physical inspection result versus a textual description — it's purely the keyword match that does the work.
- **What the code misses:** The `condition_assessment` block in the document schema — with explicit `as_found_condition` and `as_left_condition` enumerations (acceptable/degraded/failed/unknown) — is never used in Stage 4. The structured condition data is richer and more reliable than keyword detection in free text, but the retriever doesn't access it.

SOPs (Standard Operating Procedures)

- **What they are:** Prescriptive documents describing how things should work and what actions operators should take. They contain diagnostic logic, acceptance criteria, and step-by-step guidance.
- **What they uniquely provide:** Engineering knowledge about the *expected* behavior of systems and the *diagnostic rules* that distinguish failure modes. SOP-U2-CND-001 Step 4.2 in the test case says "DO above 20 ppb is indicative of air in-leakage — tube fouling does not cause dissolved oxygen elevation." That is an expert diagnostic rule, not an observation.
- **What the code misses:** The system treats this diagnostic rule the same as any other supporting snippet. It scores as contextual or supporting for air in-leakage because it contains "air in-leakage" near positive language. But its real value is as a **discriminating rule** — it explicitly says fouling does NOT produce DO elevation. A document-type-aware system would recognize SOP diagnostic rules as having special standing: they are the engineering basis for distinguishing between hypotheses, not just additional evidence for one of them.

SOPs also carry a temporal peculiarity — they describe steady-state expected behavior, not event-specific observations. The code applies the same temporal relevance logic to SOPs as to CRs, which is conceptually wrong.

ECAs (Engineering Cause Analyses)

- **What they are:** Formal engineering documents that present a structured causal argument, often with an explicitly stated confirmed root cause, contributing factors, evidence items, and recommended corrective actions.
- **What they uniquely provide:** The closest thing to a confirmed causal conclusion in the document corpus. An ECA is more epistemically reliable than a CR (which is preliminary) and more specific than an FMEA (which is generic). ECA-2022-1103 in the test case explicitly states "root cause confirmed as tube fouling" with "DO was within normal limits during that event at 7.2 ppb" — that last detail is the key discriminating fact that distinguishes the prior event from the current one.
- **What the code misses:** It doesn't recognize that an ECA's root cause statement is a confirmed finding with a higher confidence level than a CR's preliminary assessment. It also doesn't parse the ECA's `causal_factors` array or `evidence_items` array — these structured fields in the document schema contain exactly the kind of machine-readable causal knowledge the system needs, but the retriever works only on embedded text snippets.

RCA (Root Cause Analysis reports)

- **What they are:** Comprehensive post-event analyses, similar to ECAs but typically more extensive, including event timelines, causal factor charts, barrier analyses, and effectiveness reviews.
- **What they uniquely provide:** Confirmed causal chains with explicit evidence linkage. They also often contain the *negative findings* — what was ruled out and why — which are directly useful for hypothesis discrimination.
- **What the code misses:** Same as ECA — the confirmed causal status of the findings is not distinguished from a preliminary CR observation. The ruling-out logic (what was eliminated and the evidence that eliminated it) is particularly valuable and completely invisible to the current keyword-based classifier.

FMEAs (Failure Mode and Effects Analyses)

- **What they are:** Engineering design documents that enumerate possible failure modes for components, their effects, detection methods, and risk priority numbers.
- **What they uniquely provide:** They define the *possibility space* — what failures are physically conceivable for each component. They're not observations of what happened; they're engineering predictions of what could happen.
- **What the code misses:** The FMEA is already partially absorbed into the KG through the `failure_modes` nodes — so its structural content is already influencing Stage 3. When a FMEA snippet is retrieved in Stage 4, it's effectively double-counting information already present in the KG, but the system doesn't know this. A FMEA saying "air in-leakage is a known failure mode for expansion joints" scores as supporting evidence for the air in-leakage hypothesis, when it really just confirms that the hypothesis is physically plausible — something Stage 3 already established.

The right architecture would recognize that each document type occupies a different position on two axes:

Observation vs. prescription: CRs and WOs describe what was observed. SOPs and FMEAs describe what should or could happen. ECAs and RCAs describe what was confirmed.

Temporal specificity: CRs, WOs, and ECAs are event-specific. SOPs, FMEAs, and manuals are time-independent engineering knowledge.

A properly differentiated Stage 4 would use these distinctions to:

- Treat confirmed findings in ECAs and RCAs as higher-confidence evidence than preliminary observations in CRs
- Use SOP diagnostic rules as discriminating logic rather than supporting evidence for individual hypotheses
- Use FMEA content only to confirm physical plausibility, not as independent supporting evidence (since that's already captured in Stage 3)
- Weight WO condition assessments by their structured `as_found_condition` and `as_left_condition` fields rather than keyword detection
- Apply temporal discounting to CR preliminary assessments when later ECAs or RCAs have superseded them

What's Working Well

The `refine_with_evidence` method in v32 is architecturally correct — using retrieved evidence to update the prior evidence score and rerank candidates is exactly the right two-pass design. The `_eligible_review_alternative` logic that rescues near-threshold candidates when only one candidate passes is also good; it prevents the pathological case where a low evidence threshold eliminates all alternatives and the analyst gets a single candidate with no context.

The `TSKRTemporalScorerV1` latency alignment model is more principled than most heuristic temporal scorers. The separation of `latency_alignment_score` from `temporal_contradiction` is correct — something can be partially misaligned without being contradicted, and the synthesizer's confidence calibration handles this distinction properly.

The `py2neo_workflow.py` wrapper with its batch upsert design and safe token validation is solid infrastructure. The label-grouping approach in `upsert_nodes_batch` and `upsert_edges_batch` is efficient and correct.

Priority Recommendations Before Next Iteration

Ranked by impact on RCA quality:

1. Fix governance scoring — make it candidate-specific, not asset-wide
2. Add symptom-to-failure-mode structural matching using `symptom_types` and `anomaly_pattern`
3. Fix the TSKR index to handle multiple patterns per target
4. Fix the `causes` edge overloading in the KG builder
5. Make `component_ids` filtering functional in Chroma or document explicitly that it's a no-op
6. Incorporate `time_distance_days` into evidence prior scoring